

Program Analysis 2025-26

Lecture #10

Incorrectness separation logic



UNIVERSITÀ
DI PISA

SL: recap

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$t := x;$

$n := [t];$

while $n \neq \text{nil}$ do (

$t := n;$

$n := [t];$

)

$[t] := y;$

$\{\text{list}(x)\}$

Precondition

x and y are pointers to the heads of two (singly linked) lists.
The list pointed to by x is non-empty and each list is finite and null-terminated.
The two lists are disjoint (no node belongs to both lists).

The code appends the list pointed to by y
to the end of the non-empty list pointed to by x

Postcondition

A unique (singly linked) list pointed to by x .

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$
 $\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$t := x;$

$n := [t];$

while $n \neq \text{nil}$ do (

$t := n;$

$n := [t];$

)

$[t] := y;$

$\{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$
 $\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$t := x;$

$n := [t];$

while $n \neq \text{nil}$ do (

$t := n;$

$n := [t];$

)

$[t] := y;$

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$ $\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$
 $\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$n := [t];$

while $n \neq \text{nil}$ do (

$t := n;$

$n := [t];$

)

$[t] := y;$

$\{\text{ls}(x, t) * t \mapsto y\}$

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

idle frame

$R_1 \equiv \text{list}(y)$

frame

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$
 $\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$ // Ordinary Floyd's axiom

$\{x = t \wedge t \mapsto v * \text{list}(v)\} // \equiv \{ \exists t'. t = x \wedge x \mapsto v * \text{list}(v) \}$

idle frame

$R_1 \equiv \text{list}(y)$

frame

$\{\text{ls}(x, t) * t \mapsto y\}$

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell. a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$ // small axiom

idle frame

$R_1 \equiv \text{list}(y)$

frame

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

frame $\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{x = t \wedge t \mapsto v * \text{list}(v) \wedge n = v\}$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

frame

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

frame $\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{x = t \wedge t \mapsto v * \text{list}(v) \wedge n = v\}$

while $n \neq \text{nil}$ do (

$t := n;$

$n := [t];$

)

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

frame

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{x = t \wedge t \mapsto v * \text{list}(v) \wedge n = v\} \Rightarrow \{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

while $n \neq \text{nil}$ do (

$t := n;$

$n := [t];$

)

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

frame

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

frame

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

while $n \neq \text{nil}$ do (

invariant?

$t := n;$

$n := [t];$

)

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

while $n \neq \text{nil}$ do (

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n \neq \text{nil}\}$

$t := n;$

$n := [t];$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\} \)$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\}$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

invariant?

frame

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

while $n \neq \text{nil}$ do (

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n \neq \text{nil}\} \equiv \{ \exists w . \text{ls}(x, t) * t \mapsto n * n \mapsto w * \text{list}(w) \}$

$t := n;$

$n := [t];$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\})$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\}$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

list(n) is non-empty!

frame

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

frame

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

while $n \neq \text{nil}$ do (

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n \neq \text{nil}\} \equiv \{ \exists w . \text{ls}(x, t) * t \mapsto n * n \mapsto w * \text{list}(w) \}$

$t := n;$ // Ordinary Floyd's axiom

$\{ \exists t', w . \text{ls}(x, t') * t' \mapsto t * t \mapsto w * \text{list}(w) \wedge t = n \}$

$n := [t];$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\})$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\}$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

frame

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

while $n \neq \text{nil}$ do (

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n \neq \text{nil}\} \equiv \{\exists w. \text{ls}(x, t) * t \mapsto n * n \mapsto w * \text{list}(w)\}$

$t := n;$

$\{\exists t', w. \text{ls}(x, t') * t' \mapsto t * t \mapsto w * \text{list}(w) \wedge t = n\} \Rightarrow \{\exists w. \text{ls}(x, t) * t \mapsto w * \text{list}(w)\}$

$n := [t];$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\} \)$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\}$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

frame

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell. a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

while $n \neq \text{nil}$ do (

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n \neq \text{nil}\} \equiv \{ \exists w . \text{ls}(x, t) * t \mapsto n * n \mapsto w * \text{list}(w) \}$

$t := n;$

$\{ \exists w . \text{ls}(x, t) * t \mapsto w * \text{list}(w) \}$

$n := [t];$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\} \)$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\}$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

frame

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

while $n \neq \text{nil}$ do (

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n \neq \text{nil}\} \equiv \{ \exists w . \text{ls}(x, t) * t \mapsto n * n \mapsto w * \text{list}(w) \}$

$t := n;$

$\{ \exists w . \text{ls}(x, t) * t \mapsto w * \text{list}(w) \}$

$\{t \mapsto w\} \ n := [t]; \ \{t \mapsto w \wedge n = w\}$ // small axiom

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$)

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\}$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

frame

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

invariant!

while $n \neq \text{nil}$ do (

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n \neq \text{nil}\} \equiv \{\exists w. \text{ls}(x, t) * t \mapsto n * n \mapsto w * \text{list}(w)\}$

$t := n;$

$\{\exists w. \text{ls}(x, t) * t \mapsto w * \text{list}(w)\}$

$\{t \mapsto w\} \ n := [t]; \ \{t \mapsto w \wedge n = w\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\} \) \ // \ \{\exists w. \text{ls}(x, t) * t \mapsto w * \text{list}(w) \wedge n = w\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\}$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

idle frame

$R_3 \equiv \text{ls}(x, t) * \text{list}(w)$

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell. a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

invariant!

while $n \neq \text{nil}$ do (

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n \neq \text{nil}\} \equiv \{\exists w. \text{ls}(x, t) * t \mapsto n * n \mapsto w * \text{list}(w)\}$

$t := n;$

$\{\exists w. \text{ls}(x, t) * t \mapsto w * \text{list}(w)\}$

$\{t \mapsto w\} \ n := [t]; \ \{t \mapsto w \wedge n = w\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\} \) \ // \ \{\exists w. \text{ls}(x, t) * t \mapsto w * \text{list}(w) \wedge n = w\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\}$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

idle frame

$R_3 \equiv \text{ls}(x, t) * \text{list}(w)$

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell. a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

invariant!

while $n \neq \text{nil}$ do (

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n \neq \text{nil}\} \equiv \{\exists w. \text{ls}(x, t) * t \mapsto n * n \mapsto w * \text{list}(w)\}$

$t := n;$

$\{\exists w. \text{ls}(x, t) * t \mapsto w * \text{list}(w)\}$

$\{t \mapsto w\} \ n := [t]; \ \{t \mapsto w \wedge n = w\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$)

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\}$

$\{t \mapsto _ \} \ [t] := y; \ \{t \mapsto y\}$ // small axiom

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell. a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

idle frame

$R_3 \equiv \text{ls}(x, t) * \text{list}(w)$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

invariant!

while $n \neq \text{nil}$ do (

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n \neq \text{nil}\} \equiv \{\exists w. \text{ls}(x, t) * t \mapsto n * n \mapsto w * \text{list}(w)\}$

$t := n;$

$\{\exists w. \text{ls}(x, t) * t \mapsto w * \text{list}(w)\}$

$\{t \mapsto w\} \ n := [t]; \ \{t \mapsto w \wedge n = w\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$)

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\} \Rightarrow \{\text{ls}(x, t) * t \mapsto _ \}$

$\{t \mapsto _ \} \ [t] := y; \ \{t \mapsto y\}$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

idle frame

$R_3 \equiv \text{ls}(x, t) * \text{list}(w)$

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell. a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$\{t \mapsto v\} \ n := [t]; \ \{t \mapsto v \wedge n = v\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

invariant!

while $n \neq \text{nil}$ do (

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n \neq \text{nil}\} \equiv \{\exists w. \text{ls}(x, t) * t \mapsto n * n \mapsto w * \text{list}(w)\}$

$t := n;$

$\{\exists w. \text{ls}(x, t) * t \mapsto w * \text{list}(w)\}$

$\{t \mapsto w\} \ n := [t]; \ \{t \mapsto w \wedge n = w\}$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$)

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\} \Rightarrow \{\text{ls}(x, t) * t \mapsto _ \}$

$\{t \mapsto _ \} \ [t] := y; \ \{t \mapsto y\}$

$\{\text{ls}(x, t) * t \mapsto y\}$

$\{\text{ls}(x, t) * t \mapsto y * \text{list}(y)\} \Rightarrow \{\text{list}(x)\}$

idle frame

$R_1 \equiv \text{list}(y)$

idle frame

$R_2 \equiv x = t \wedge \text{list}(v)$

idle frame

$R_3 \equiv \text{ls}(x, t) * \text{list}(w)$

idle frame

$R_4 \equiv \text{ls}(x, t)$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell. a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

Example: concatenation

$\{x \mapsto v * \text{list}(v) * \text{list}(y)\}$

$\{x \mapsto v * \text{list}(v)\}$

$t := x;$

$\{x = t \wedge t \mapsto v * \text{list}(v)\}$

$n := [t];$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\}$

invariant!

while $n \neq \text{nil}$ do (

$t := n;$

$n := [t];$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n)\})$

$\{\text{ls}(x, t) * t \mapsto n * \text{list}(n) \wedge n = \text{nil}\} \Rightarrow \{\text{ls}(x, t) * t \mapsto _ \}$

$[t] := y;$

$\{\text{ls}(x, t) * t \mapsto y\}$

$\{\text{list}(x)\}$

$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell . a_1 \mapsto \ell * \text{ls}(\ell, a_2))$

$\text{list}(a) \triangleq \text{ls}(a, \text{nil})$

SL: correctness and (in)completeness

Relational semantics

$$\llbracket \text{skip} \rrbracket \triangleq \{(\sigma, \sigma)\}$$

$$\llbracket b? \rrbracket \triangleq \{(\sigma, \sigma) \mid \sigma = \langle s, h \rangle \wedge s \models b\}$$

$$\llbracket x := a \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto \llbracket a \rrbracket s], h)\}$$

$$\llbracket x := [a] \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto v], h \rangle) \mid v = h(\llbracket a \rrbracket s) \in \mathbb{Z}\}$$

$$\llbracket [a_1] := a_2 \rrbracket \triangleq \{(\langle s, h \rangle, \langle s, h[\llbracket a_1 \rrbracket s \mapsto \llbracket a_2 \rrbracket s] \rangle) \mid h(\llbracket a_1 \rrbracket s) \in \mathbb{Z}\}$$

$$\llbracket x := \text{alloc}() \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto n], h[n \mapsto v] \rangle) \mid v \in \mathbb{Z} \wedge (n \notin \text{dom}(h) \vee h(n) = \perp)\}$$

$$\llbracket \text{free}(x) \rrbracket \triangleq \{(\langle s, h \bullet [s(x) \mapsto v] \rangle, \langle s, h \rangle) \mid s(x) \in \mathbb{N} \wedge v \in \mathbb{Z}\}$$

$$\begin{aligned} \llbracket x := \text{cons}(a_0, \dots, a_k) \rrbracket \triangleq & \{(\langle s, h \rangle, \langle s[x \mapsto n], h[n \mapsto \llbracket a_0 \rrbracket s, \dots, (n+k) \mapsto \llbracket a_k \rrbracket s] \rangle) \\ & \mid (\forall i \in [n, n+k]. i \notin \text{dom}(h) \vee h(i) = \perp)\} \end{aligned}$$

Correctness + Incompleteness

Th. *[correctness]*

If $\{P\} c \{Q\}$ then $\llbracket c \rrbracket P \subseteq Q$

Proof. By induction on the derivation.

Th. *[incompleteness]*

There exist valid SL triples that are not provable

Proof. Misses footprint theorem: coming next!

Final considerations on SL

Simple Proofs for Simple Programs

SL addresses resource manipulation

separating conjunction for in-place reasoning

pre/post describe local surgeries

Exercises

Satisfaction 1

Can you find some state that satisfies the following assertions?

$$(x \dot{=} y) * x \mapsto y$$

$$a_1 \dot{=} a_2 \triangleq (a_1 = a_2) \wedge \text{emp}$$

$$(x = y) * x \mapsto y$$

$$(x = y) \wedge x \mapsto y$$

$$x \mapsto x$$

Satisfaction 1

Can you find some state that satisfies the following assertions?

$$(x \dot{=} y) * x \mapsto y$$



$$a_1 \dot{=} a_2 \triangleq (a_1 = a_2) \wedge \text{emp}$$

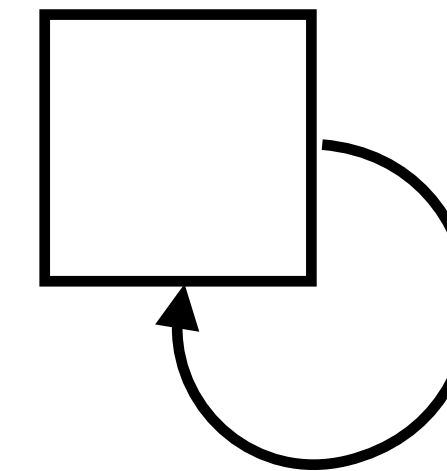
$$(x = y) * x \mapsto y$$



$$(x = y) \wedge x \mapsto y$$



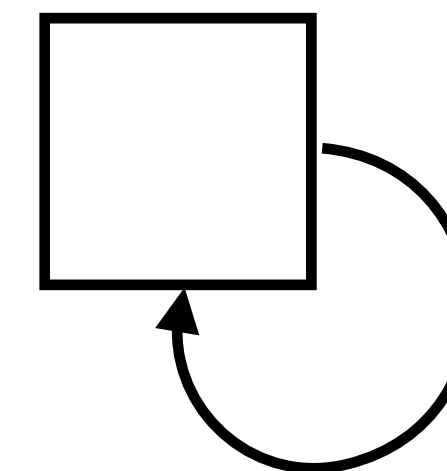
$x \quad y$



$$x \mapsto x$$



x



Satisfaction 2

Show a state that satisfies the assertion $(x \mapsto y) * \neg(x \mapsto y)$

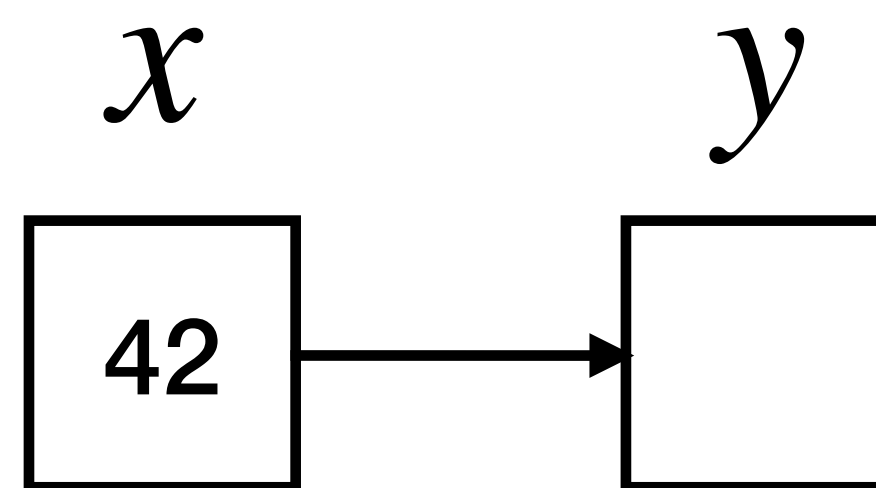
Satisfaction 2

Show a state that satisfies the assertion $(x \mapsto y) * \neg(x \mapsto y)$

$$\text{dom}(h) = \{11\}$$

$$s(x) = 11 \quad h(11) = 42$$

$$s(y) = 42$$



List segments

Consider the *imprecise list segment* definition below

$$\text{ils}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (\exists v. a_1 \mapsto v * \text{ils}(v, a_2))$$

Prove that $\text{ils}(a_1, a_2) \not\equiv \text{ls}(a_1, a_2)$, by finding a state that satisfies $\text{ils}(42, 42)$ but not $\text{ls}(42, 42)$

$$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists v. a_1 \mapsto v * \text{ls}(v, a_2))$$

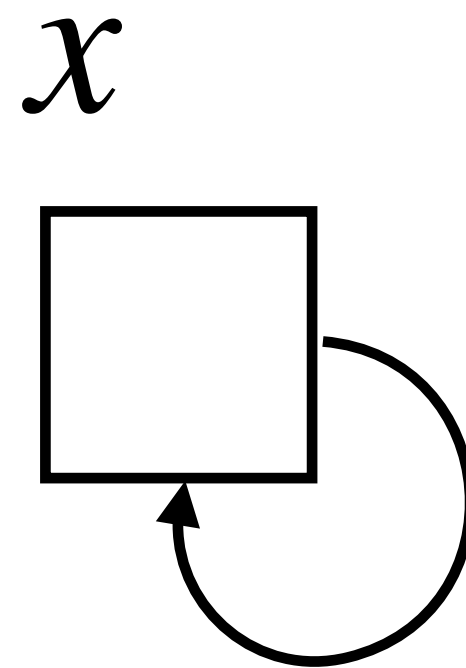
List segments

Consider the *imprecise list segment* definition below

$$\text{ils}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (\exists v. a_1 \mapsto v * \text{ils}(v, a_2))$$

Prove that $\text{ils}(a_1, a_2) \not\equiv \text{ls}(a_1, a_2)$, by finding a state that satisfies $\text{ils}(42, 42)$ but not $\text{ls}(42, 42)$

$$\begin{aligned} \text{dom}(h) &= \{42\} \\ s(x) &= 42 \quad h(42) = 42 \end{aligned}$$



$$\langle s, h \rangle \not\models \text{ls}(42, 42)$$

$$\langle s, h \rangle \models \text{ils}(42, 42)$$

$$\text{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists v. a_1 \mapsto v * \text{ls}(v, a_2))$$

Footprint property

Footprint recipe

Given a command c

1. Derive the specification of c using **local axioms**

$$\begin{array}{l} \{x \mapsto _ \} \\ \quad [x] := 1; \\ \{x \mapsto 1 \} \\ \quad y := [x] \\ \{x \mapsto 1 \wedge y = 1 \} \end{array}$$

Footprint recipe

Given a command c

1. Derive the specification of c using **local axioms**
2. Apply rule frame to complete the specification

frame

$$\begin{array}{l} \{R * x \mapsto _ \} \\ \{x \mapsto _ \} \\ [x] := 1; \\ \{x \mapsto 1 \} \\ y := [x] \\ \{x \mapsto 1 \wedge y = 1 \} \\ \{R * x \mapsto 1 \wedge y = 1 \} \end{array}$$

Footprint fails in SL

Counterexample:

We would like to derive

$$\{y \mapsto _ \} x := \text{alloc}(); \text{free}(x) \{y \mapsto _ \wedge y \neq x\}$$

$\{\text{emp}\}$

$x := \text{alloc}();$

$\{ \exists v. v \mapsto _ \wedge x = v \}$

$\text{free}(x)$

$\{ \exists v. x = v \}$

Footprint fails in SL

Counterexample:

We would like to derive

$$\{y \mapsto _ \} x := \text{alloc}(); \text{free}(x) \{y \mapsto _ \wedge y \neq x\}$$

$\{y \mapsto _ * \text{emp}\}$

$\{\text{emp}\}$

$x := \text{alloc}();$

$\{\exists v. v \mapsto _ \wedge x = v\}$

$\text{free}(x)$

$\{\exists v. x = v\}$

$\{\exists v. y \mapsto _ \wedge x = v\}$ // strongest derivable spec: **incomplete spec**

lossy: x held a location!

$\{x \mapsto _ \} \text{free}(x) \{\text{emp}\}$

Information loss

$\{x \mapsto _ \} [x] := y \{x \mapsto y\}$ // write

$\{y \mapsto v\} x := [y] \{x = v \wedge y \mapsto v\}$ // read

resources can grow

$\{\text{emp}\} x := \text{alloc}() \{x \mapsto _ \}$ // alloc

$\{x \mapsto _ \} \text{free}(x) \{\text{emp}\}$ // dispose

but should never shrink!

Incorrectness Separation Logic (ISL)

CAV 2020

Local Reasoning About the Presence of Bugs: Incorrectness Separation Logic

Azalea Raad^{1(✉)}, Josh Berdine², Hoang-Hai Dang¹, Derek Dreyer¹,
Peter O'Hearn^{2,3}, and Jules Villard²

¹ Max Planck Institute for Software Systems (MPI-SWS),
Kaiserslautern and Saarbrücken, Germany
{azalea,haidang,dreyer}@mpi-sws.org

² Facebook, London, UK
{jjb,peteroh,jul}@fb.com

³ University College London, London, UK

Abstract. There has been a large body of work on local reasoning for proving the *absence* of bugs, but none for proving their *presence*. We present a new formal framework for local reasoning about the presence of bugs, building on two complementary foundations: 1) separation logic and 2) incorrectness logic. We explore the theory of this new *incorrectness separation logic* (ISL), and use it to derive a begin-anywhere, intra-procedural symbolic execution analysis that has no false positives *by construction*. In so doing, we take a step towards transferring modular, scalable techniques from the world of program verification to bug catching.

Keywords: Program logics · Separation logic · Bug catching

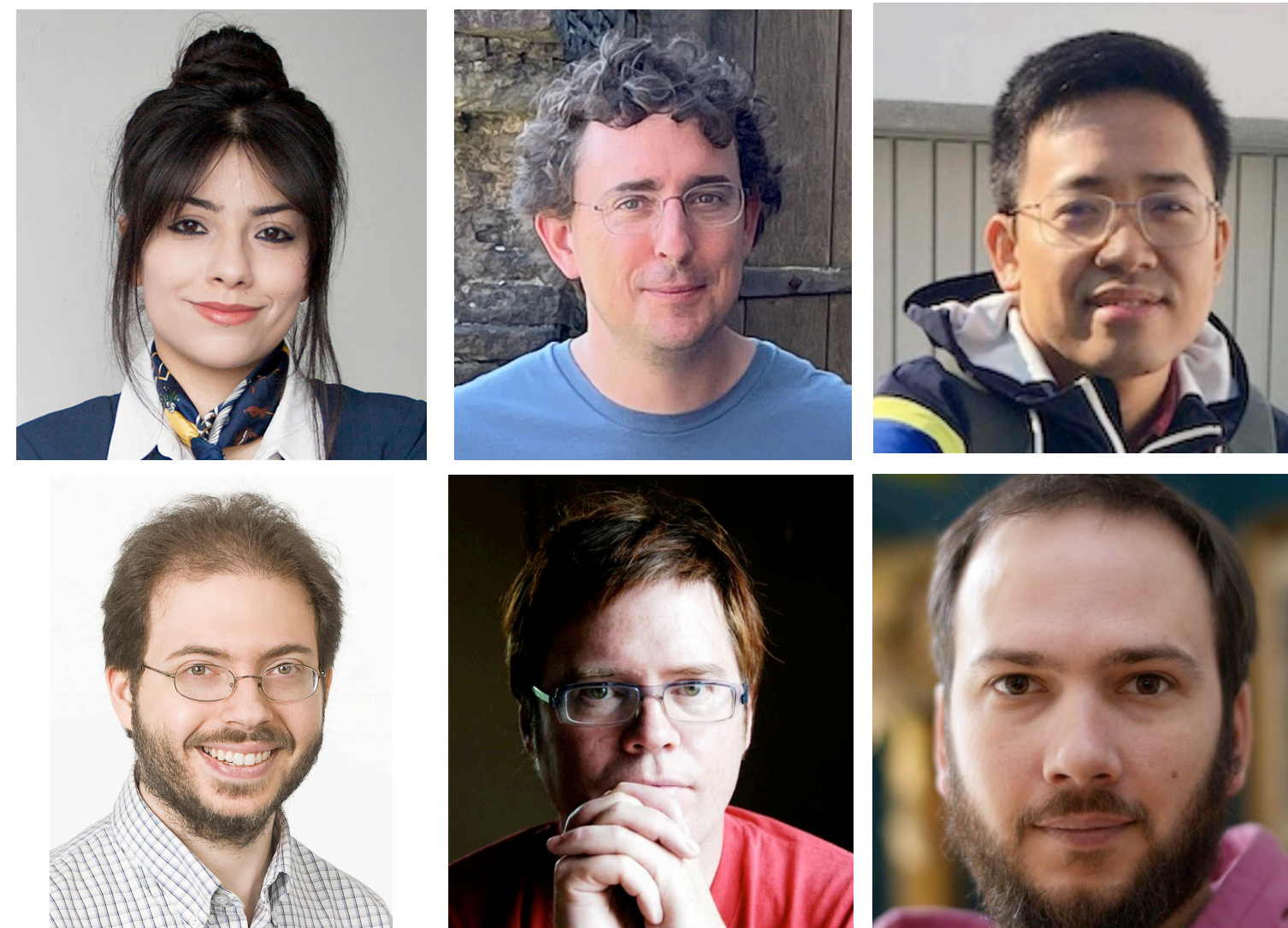
1 Introduction

There has been significant research on sound, local reasoning about the state for proving the absence of bugs (e.g., [2, 13, 26, 29, 30, 41]). Locality leads to techniques that are compositional *both* in code (concentrating on a program component) and in the resources accessed (spatial locality), without tracking the entire global state or the global program within which a component sits. Compositionality enables reasoning to scale to large teams and codebases: reasoning can be done even when a global program is not present (e.g., a library, or during program construction), without having to write the analogue of a test or verification harness, and the results of reasoning about components can be composed efficiently [11].

Meanwhile, many of the practical applications of symbolic reasoning have aimed at proving the *presence* of bugs (i.e., bug catching), rather than proving their absence (i.e., correctness). Logical bug catching methods include symbolic model checking [7, 12] and symbolic execution for testing [9]. These methods are usually formulated as global analyses; but, the rationale of local reasoning holds just as well for bug catching as it does for correctness: it has the potential to

© The Author(s) 2020
S. K. Lahiri and C. Wang (Eds.): CAV 2020, LNCS 12225, pp. 225–252, 2020.
https://doi.org/10.1007/978-3-030-53291-8_14

“bugs are a fundamental enough phenomenon to warrant a fundamental compositional theory for reasoning positively about their existence”



ISL = IL + SL

bug finding

IL

$$[P] \text{ } \mathcal{C} \text{ } [\epsilon : Q]$$

local reasoning

SL

$$\{P\} \text{ } \mathcal{C} \text{ } \{Q\}$$
$$\frac{\{P\} \text{ } \mathcal{C} \text{ } \{Q\}}{\{P * R\} \text{ } \mathcal{C} \text{ } \{Q * R\}}$$

ISL

$$[P] \text{ } \mathcal{C} \text{ } [\epsilon : Q]$$
$$\frac{[P] \text{ } \mathcal{C} \text{ } [\epsilon : Q]}{[P * R] \text{ } \mathcal{C} \text{ } [\epsilon : Q * R]}$$

Regular commands

regular
command

$c ::=$

e

atomic
command

$c_1; c_2$

choice

$c_1 + c_2$

$c^\star$

Kleene
star

$e ::=$ skip

$b?$

$x := a$

simplified
($x \neq y$)

$x := [y]$ // read

$[x] := y$ // write

$x := \text{alloc}()$

$\text{free}(x)$

can fail

$\text{error}()$

...

Disclaim

for the sake of presentation, some
technical aspects are not fully detailed



Local axioms: write

$$\text{SL} \quad \frac{}{\{x \mapsto _ \} [x] := y \{x \mapsto y \}}$$

$$[x \mapsto v] [x] := y \text{ [ok : } x \mapsto y \text{]}$$

must put a
value

$$[x = \text{nil}] [x] := y \text{ [er : } x = \text{nil} \text{]}$$

null pointer
dereference

Remind: reversed consequence

{HL}

$$P \Rightarrow P' \quad \{P'\} c \{Q'\} \quad Q' \Rightarrow Q$$

$$\{P\} c \{Q\}$$

preconditions
can be
strengthened

local axioms
can speak
broadly

$$\{x \mapsto _ \} [x] := y \{x \mapsto y\}$$

[IL]

$$P' \Rightarrow P \quad [P'] c [Q'] \quad Q \Rightarrow Q'$$

$$[P] c [Q]$$

preconditions
can be
weakened

local axioms
must speak
precisely

$$[x \mapsto v] [x] := y [x \mapsto y]$$

Local axioms: read

SL

$$\frac{}{\{y \mapsto v\} x := [y] \{x = v \wedge y \mapsto v\}}$$

$$[y \mapsto v] x := [y] [\text{ok} : x = v \wedge y \mapsto v]$$

$$[y = \text{nil}] x := [y] [\text{er} : y = \text{nil}]$$

null pointer
dereference

Local axioms: allocation

SL

$$\frac{}{\{\text{emp}\} \ x := \text{alloc}() \ \{x \mapsto _ \}}$$

$$[x \dot{=} x'] \ x := \text{alloc}() \ [\text{ok} : x \mapsto _]$$

needed for
footprint

Local axioms: dispose (1st try)

SL

$$\frac{}{\{x \mapsto _ \} \text{ free}(x) \{ \text{emp} \}}$$

$$[x \mapsto v] \text{ free}(x) [\text{ok} : \text{emp}]$$

must put a
value

$$[x = \text{nil}] \text{ free}(x) [\text{er} : x = \text{nil}]$$

null pointer
dereference

Unsound Frame rule!

$$\frac{[P] \text{ c } [\epsilon : Q]}{[P * R] \text{ c } [\epsilon : Q * R]}$$

$$\frac{\frac{\frac{}{[x \mapsto v] \text{ free}(x) [\text{ok} : \text{emp}]} [\text{free}]}{[x \mapsto v * x \mapsto v] \text{ free}(x) [\text{ok} : \text{emp} * x \mapsto v]} [\text{frame}]}{[\text{false}] \text{ free}(x) [\text{ok} : x \mapsto v]} [\text{cons}]$$

the only sound
under approximation
can be false

this can be fixed by using a
monotonic heap model
(and we will recover
completeness)

Solution

assertion

$P ::=$ true | false | $a_1 < a_2$ | $a_1 = a_2$ | ...
| $\neg P$ | $P_1 \wedge P_2$ | $\exists x. P$ | ...
| emp
| $a_1 \mapsto a_2$
| $P_1 * P_2$
| $x \nrightarrow$

Boolean and
classical
assertions

structural
assertions

track deallocated
locations

Satisfaction: structural

$\langle s, h \rangle \models a_1 \mapsto a_2$ iff $\text{dom}(h) = \{ \llbracket a_1 \rrbracket s \}$ and $h(\llbracket a_1 \rrbracket s) = \llbracket a_2 \rrbracket s$

$\langle s, h \rangle \models \text{emp}$ iff h is the empty map $[]$

$\langle s, h \rangle \models P_1 * P_2$ iff $\exists h_1, h_2 . \langle s, h_1 \rangle \models P_1$ and $\langle s, h_2 \rangle \models P_2$ and $h = h_1 \bullet h_2$

$\langle s, h \rangle \models x \nrightarrow$ iff $\text{dom}(h) = \{ s(x) \}$ and $h(s(x)) = \perp$

Notably: $x \mapsto v * x \nrightarrow \equiv \text{false} \equiv x \nrightarrow * x \nrightarrow$
 $y \mapsto v * x \nrightarrow \equiv y \mapsto v * x \nrightarrow \wedge x \neq y$

Local axioms: dispose

SL

$$\frac{}{\{x \mapsto _ \} \text{ free}(x) \{ \text{emp} \}}$$

$$[x \mapsto v] \text{ free}(x) [\text{ok} : x \not\mapsto _]$$

must put a
value

track deallocated
locations

this way resources
cannot shrink

$$[x = \text{nil}] \text{ free}(x) [\text{er} : x = \text{nil}]$$

null pointer
dereference

Example

$$\frac{\frac{\frac{[x \mapsto v] \text{ free}(x) \text{ [ok : } x \not\mapsto \text{]}}{\text{[free]}}}{[x \mapsto v * x \mapsto v] \text{ free}(x) \text{ [ok : } x \not\mapsto * x \mapsto v \text{]}}}{\text{[false] free}(x) \text{ [ok : false]}} \frac{\text{[frame]}}{\text{[cons]}}$$

sound
under approximation!

Footprint holds in ISL

We would like to derive

$$[y \mapsto _] \ x := \text{alloc}() ; \text{free}(x) \ [\text{ok} : y \mapsto _ \wedge x \not\mapsto _ \wedge y \neq x]$$

$$[y \mapsto _ * \text{emp}]$$

$$[\text{emp}]$$

$$x := \text{alloc}();$$

$$[x \mapsto v]$$

$$\text{free}(x)$$

$$[x \not\mapsto _]$$

$$[\text{ok} : y \mapsto _ * x \not\mapsto _ \wedge y \neq x]$$

Additional local axioms

$$[x \not\mapsto] [x] := y [er : x \not\mapsto]$$

use after free
errors

$$[y \not\mapsto] x := [y] [er : y \not\mapsto]$$

$$[x \not\mapsto] \text{free}(x) [er : x \not\mapsto]$$

double free
error

$$[y \not\mapsto] x := \text{alloc}() [ok : x \mapsto v \wedge x = y]$$

reuse of
deallocated
locations

Soundness and completeness

Relational semantics

$$\llbracket \text{skip} \rrbracket_{\text{ok}} \triangleq \{(\sigma, \sigma)\}$$

$$\llbracket b? \rrbracket_{\text{ok}} \triangleq \{(\sigma, \sigma) \mid \sigma = \langle s, h \rangle \wedge s \models b\}$$

$$\llbracket x := a \rrbracket_{\text{ok}} \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto \llbracket a \rrbracket s], h)\}$$

$$\llbracket \text{error}() \rrbracket_{\text{ok}} \triangleq \emptyset$$

$$\llbracket \text{skip} \rrbracket_{\text{er}} \triangleq \emptyset$$

$$\llbracket b? \rrbracket_{\text{er}} \triangleq \emptyset$$

$$\llbracket x := a \rrbracket_{\text{er}} \triangleq \emptyset$$

$$\llbracket \text{error}() \rrbracket_{\text{er}} \triangleq \{(\sigma, \sigma)\}$$

$$\llbracket x := [y] \rrbracket_{\text{ok}} \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto v], h \rangle) \mid v = h(s(y)) \in \mathbb{Z}\}$$

$$\llbracket x := [y] \rrbracket_{\text{er}} \triangleq \{(\langle s, h \rangle, \langle s, h \rangle) \mid s(y) = \text{nil} \vee h(s(y)) = \perp\}$$

$$\llbracket [x] := y \rrbracket_{\text{ok}} \triangleq \{(\langle s, h \rangle, \langle s, h[s(x) \mapsto s(y)] \rangle) \mid h(s(x)) \in \mathbb{Z}\}$$

$$\llbracket [x] := y \rrbracket_{\text{er}} \triangleq \{(\langle s, h \rangle, \langle s, h \rangle) \mid s(x) = \text{nil} \vee h(s(x)) = \perp\}$$

$$\llbracket x := \text{alloc}() \rrbracket_{\text{ok}} \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto n], h[n \mapsto v] \rangle) \mid v \in \mathbb{Z} \wedge (n \notin \text{dom}(h) \vee h(n) = \perp)\}$$

$$\llbracket x := \text{alloc}() \rrbracket_{\text{er}} \triangleq \emptyset$$

$$\llbracket \text{free}(x) \rrbracket_{\text{ok}} \triangleq \{(\langle s, h \bullet [s(x) \mapsto v] \rangle, \langle s, h \bullet [s(x) \mapsto \perp] \rangle) \mid s(x) \in \mathbb{N} \wedge v \in \mathbb{Z}\}$$

$$\llbracket \text{free}(x) \rrbracket_{\text{er}} \triangleq \{(\langle s, h \rangle, \langle s, h \rangle) \mid s(x) = \text{nil} \vee h(s(x)) = \perp\}$$

Actual ISL rules

SKIP $\vdash [\mathbf{emp}] \text{ skip } [ok : \mathbf{emp}]$	ASSIGN $\vdash [x=x'] x := e [ok : x=e[x'/x]]$	HAVOC $\vdash [x=x'] x := * [ok : x=v]$		
ASSUME $\vdash [\mathbf{emp}] \text{ assume}(B) [ok : B]$	ERROR $\vdash [\mathbf{emp}] \text{ L: error } [er(L) : \mathbf{emp}]$	FRAME $\frac{\vdash [p] \mathbb{C} [\epsilon : q] \quad \text{mod}(\mathbb{C}) \cap \text{fv}(r) = \emptyset}{\vdash [p * r] \mathbb{C} [\epsilon : q * r]}$	ALLOC1 $\vdash [x=x'] x := \text{alloc}() [ok : x \mapsto -]$	
SEQ1 $\frac{\vdash [p] \mathbb{C}_1 [er(L) : q]}{\vdash [p] \mathbb{C}_1; \mathbb{C}_2 [er(L) : q]}$	SEQ2 $\frac{\vdash [p] \mathbb{C}_1 [ok : r] \quad \vdash [r] \mathbb{C}_2 [\epsilon : q]}{\vdash [p] \mathbb{C}_1; \mathbb{C}_2 [\epsilon : q]}$	LOOP1 $\vdash [p] \mathbb{C}^* [ok : p]$	FREE $\vdash [x \mapsto e] \text{ L: free}(x) [ok : x \not\mapsto]$	ALLOC2 $\vdash [x=x' * y \not\mapsto] x := \text{alloc}() [ok : x=y * y \mapsto -]$
CHOICE $\frac{\vdash [p] \mathbb{C}_i [\epsilon : q] \quad \text{for some } i \in \{1, 2\}}{\vdash [p] \mathbb{C}_1 + \mathbb{C}_2 [\epsilon : q]}$	EXIST $\frac{\vdash [p] \mathbb{C} [\epsilon : q] \quad x \notin \text{fv}(\mathbb{C})}{\vdash [\exists x. p] \mathbb{C} [\epsilon : \exists x. q]}$	LOOP2 $\frac{\vdash [p] \mathbb{C}^*; \mathbb{C} [\epsilon : q]}{\vdash [p] \mathbb{C}^* [\epsilon : q]}$	FREEER $\vdash [x \not\mapsto] \text{ L: free}(x) [er(L) : x \not\mapsto]$	REENULL $\vdash [x=\mathbf{null}] \text{ L: free}(x) [er(L) : x=\mathbf{null}]$
CONS $\frac{p' \Rightarrow p \quad \vdash [p'] \mathbb{C} [\epsilon : q'] \quad q \Rightarrow q'}{\vdash [p] \mathbb{C} [\epsilon : q]}$	DISJ $\frac{\vdash [p_1] \mathbb{C} [\epsilon : q_1] \quad \vdash [p_2] \mathbb{C} [\epsilon : q_2]}{\vdash [p_1 \vee p_2] \mathbb{C} [\epsilon : q_1 \vee q_2]}$	LOAD $\vdash [x=x' * y \mapsto e] \text{ L: } x := [y] [ok : x=e[x'/x] * y \mapsto e[x'/x]]$	STORE $\vdash [x \mapsto e] \text{ L: } [x] := y [ok : x \mapsto y]$	
		LOADER $\vdash [y \not\mapsto] \text{ L: } x := [y] [er(L) : y \not\mapsto]$	STOREER $\vdash [x \not\mapsto] \text{ L: } [x] := y [er(L) : x \not\mapsto]$	
		LOADNULL $\vdash [y=\mathbf{null}] \text{ L: } x := [y] [er(L) : y=\mathbf{null}]$	STORENULL $\vdash [x=\mathbf{null}] \text{ L: } [x] := y [er(L) : x=\mathbf{null}]$	
SUBST $\frac{\vdash [p] \mathbb{C} [\epsilon : q] \quad y \notin \text{fv}(p, \mathbb{C}, q)}{\vdash [p[y/x]] \mathbb{C}[y/x] [\epsilon : q[y/x]]}$	LOCAL $\frac{\vdash [p] \mathbb{C} [\epsilon : q]}{\vdash [\exists x. p] \text{ local } x \text{ in } \mathbb{C} [\epsilon : \exists x. q]}$			

Correctness + footprint

Th. [*correctness*]

If $[P] \ c \ [\epsilon : Q]$ then $Q \subseteq \llbracket c \rrbracket \epsilon(P)$

Proof. By induction on the derivation.

Th. [*footprint*]

Any valid ISL triple $[\sigma_P] \ c \ [\epsilon : \sigma_Q]$ can be derived

Proof. See CAV2020 paper for details.

Final considerations on SL

**ISL = IL + SL
for bug
catching!**

ISL address compositional bug catching

targets memory safety bugs (use-after-free)

no-false-positives theorem: all bugs are true

Exercises

Soundness

Is the axiom $[x \mapsto _][x] := y [ok : x \mapsto y]$ sound?

Soundness

Is the axiom $[x \mapsto _][x] := y \text{ [ok : } x \mapsto y]$ sound?

$$\frac{(x \mapsto v) \Rightarrow (x \mapsto _) \quad \overline{[x \mapsto v][x] := y \text{ [ok : } x \mapsto y]}}{[x \mapsto _][x] := y \text{ [ok : } x \mapsto y]}$$

Unsoundness

Prove that rule [*conj] is **unsound**

$$\frac{[P_1] \text{ c } [Q_1] \quad [P_2] \text{ c } [Q_2]}{[P_1 * P_2] \text{ c } [Q_1 * Q_2]} \text{ [*conj]}$$

Unsoundness

Prove that rule [*conj] is **unsound**

$$\frac{[P_1] \text{ c } [Q_1] \quad [P_2] \text{ c } [Q_2]}{[P_1 * P_2] \text{ r } [Q_1 * Q_2]} \text{ [*conj]}$$

It is enough to consider pure assertions! $P * Q \equiv P \wedge Q$

Consider $[x = 0] \ x := 1 \ [x = 1]$ and $[x = 1] \ x := 1 \ [x = 1]$

By rule [*conj] we could derive $[false] \ x := 1 \ [x = 1]$

which is not sound!

Separation SIL

OOPSLA 2025

Revealing Sources of (Memory) Errors via Backward Analysis

FLAVIO ASCARI, University of Pisa, Italy
ROBERTO BRUNI, University of Pisa, Italy
ROBERTA GORI, University of Pisa, Italy
FRANCESCO LOGOZZO, Meta Platforms, USA

Sound over-approximation methods are effective for proving the absence of errors, but inevitably produce false alarms that can hamper programmers. In contrast, under-approximation methods focus on bug detection and are free from false alarms. In this work, we present two novel proof systems designed to locate the source of errors via backward under-approximation, namely Sufficient Incorrectness Logic (SIL) and its specialization for handling memory errors, called Separation SIL. The SIL proof system is minimal, sound and complete for Lisbon triples, enabling a detailed comparison of triple-based program logics across various dimensions, including negation, approximation, execution order, and analysis objectives. More importantly, SIL lays the foundation for our main technical contribution, by distilling the inference rules of Separation SIL, a sound and (relatively) complete proof system for automated backward reasoning in programs involving pointers and dynamic memory allocation. The completeness result for Separation SIL relies on a careful crafting of both the assertion language and the rules for atomic commands.

CCS Concepts: • **Theory of computation** → **Logic and verification**; *Proof theory*; *Hoare logic*; **Separation logic**; *Programming logic*.

Additional Key Words and Phrases: Sufficient Incorrectness Logic, Incorrectness Logic, Outcome Logic

ACM Reference Format:

Flavio Ascari, Roberto Bruni, Roberta Gori, and Francesco Logozzo. 2025. Revealing Sources of (Memory) Errors via Backward Analysis. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 127 (April 2025), 28 pages. <https://doi.org/10.1145/3720486>

1 Introduction

Formal methods aim to automate the improvement of software reliability and security. Notable success stories are, e.g., the Astrée static analyzer [Blanchet et al. 2003], the SLAM model checker [Ball and Rajamani 2001], the certified C compiler CompCert [Leroy 2009], VCC for safety properties verification [Cohen et al. 2009], and the Frama-C platform for the integration of many C code analyses [Baudin et al. 2021]. Despite that, effective program correctness methods struggle to reach mainstream adoption, mostly because they exploit over-approximation to handle decidability issues and false positives are seen as a distraction by expert programmers. Being free from false positives is possibly the reason why *under-approximation* approaches for bug-finding, such as testing and bounded model checking, are preferred in industrial applications. Incorrectness Logic (IL) [O’Hearn 2020] is a new program logic for bug-finding: *any error state found in the post can be produced by some input states that satisfy the pre*. However, IL triples are not able to characterize precisely *the input states that are responsible for a given error*. This is possibly rooted in the *forward* flavor of the under-approximation, which follows the ordinary direction of code execution.

Authors’ Contact Information: Flavio Ascari, University of Pisa, Pisa, Italy, flavio.ascari@phd.unipi.it; Roberto Bruni, University of Pisa, Pisa, Italy, bruni@di.unipi.it; Roberta Gori, University of Pisa, Pisa, Italy, roberta.gori@unipi.it; Francesco Logozzo, Meta Platforms, Seattle, USA, logozzo@meta.com.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2475-1421/2025/4-ART127

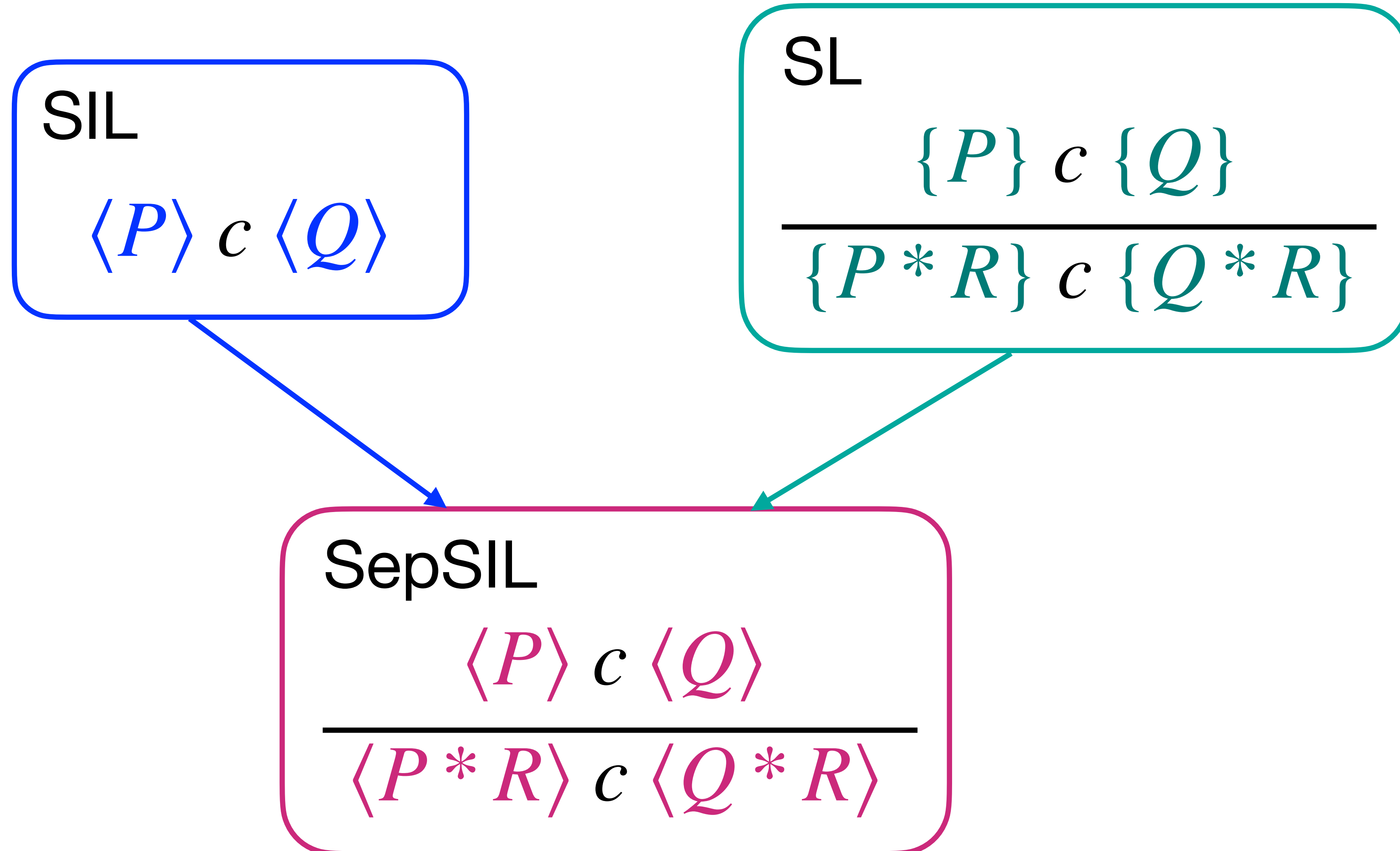
<https://doi.org/10.1145/3720486>

Proc. ACM Program. Lang., Vol. 9, No. OOPSLA1, Article 127. Publication date: April 2025.

“Separation SIL can yield more succinct postconditions and provide stronger guarantees than ISL and can support effective backward reasoning”



SepSIL = SIL + SL



Regular commands

regular
command

$c ::=$

e

atomic
command

$c_1; c_2$

choice

$c_1 + c_2$

$c^\star$

Kleene
star

$e ::=$ skip

| $b?$

| $x := a$

simplified
($x \neq y$)

| $x := [y]$ // read

| $[x] := y$ // write

| $x := \text{alloc}()$

| $\text{free}(x)$

Assertion language

assertion

$P ::=$ true | false | $a_1 < a_2$ | $a_1 = a_2$ | ...
| $\neg P$ | $P_1 \wedge P_2$ | $\exists x. P$ | ...
| emp
| $a_1 \mapsto a_2$
| $P_1 * P_2$
| $x \nrightarrow$

Boolean and
classical
assertions

structural
assertions

track deallocated
locations

Local axioms: write

SL

$$\frac{}{\{x \mapsto _ \} [x] := y \{x \mapsto y \}}$$

ISL

$$\frac{}{[x \mapsto v] [x] := y [ok : x \mapsto y]}$$

SepSIL

$$\frac{}{\langle x \mapsto _ \rangle [x] := y \langle x \mapsto y \rangle}$$

weakest pre

Local axioms: read

SL

$$\frac{}{\{y \mapsto v\} x := [y] \{x = v \wedge y \mapsto v\}}$$

ISL

$$\frac{}{[y \mapsto v] x := [y] [\text{ok} : x = v \wedge y \mapsto v]}$$

SepSIL

$$\frac{}{\langle y \mapsto v \wedge (v = x') \rangle x := [y] \langle y \mapsto v \wedge (x = x') \rangle}$$

Hoare style

applicable to any
post

Local axioms: allocation

SL

$$\frac{}{\{\text{emp}\} x := \text{alloc}() \{x \mapsto _ \}}$$

ISL

$$\frac{}{[\text{emp}] x := \text{alloc}() [\text{ok} : x \mapsto _]}$$

SepSL

$$\frac{}{\langle \text{emp} \rangle x := \text{alloc}() \langle x \mapsto _ \rangle}$$

Local axioms: dispose

SL

$$\{x \mapsto _ \} \text{ free}(x) \{ \text{emp} \}$$

ISL

$$\frac{}{[x \mapsto v] \text{ free}(x) [\text{ok} : x \not\mapsto]}$$

SepSL

$$\frac{}{\langle x \mapsto _ \rangle \text{ free}(x) \langle x \not\mapsto \rangle}$$

using cons can be
strengthened to $x \mapsto v$

Different proofs of a real bug

Use-after-lifetime bug

```
void deref_after_pb(std::vector<int> *v) {  
    int *x = &v->at(1);  
    v->push_back(42);  
    std::cout << *x << "\n"; }
```

from std::vector library, can deallocate and then reallocate v

push_back.cpp:7: error: VECTOR_INVALIDATION. accessing memory that was potentially invalidated by 'std::vector::push_back()' on line 6.

```
5.      int *x = &(v->at(1));  
6.      v->push_back(42);  
7. >    std::cout << *x << "\n"; }
```

if v is reallocated, x is invalidated

The C++ use-after-lifetime bug (above); the Pulse error message (below).

abstracted from real
occurrences at Facebook

From C++ to regular commands

$$[v \mapsto a * a \mapsto -] \text{ client}(v) \quad [er(L_{rx}) : \exists a'. v \mapsto a' * a' \mapsto - * a \not\mapsto]$$

```
void push_back(int **v)
{
  if (nondet()) {
    free(*v);
    *v = malloc(sizeof(int));
  }
}
```

```
void client(v) {
  int* x = *v;
  push_back(v);
  *x = 42; }
```

C version

```
push_back(v)  $\triangleq$ 
  local z, y in
    z := *;
    (assume(z  $\neq$  0); Lrv: y := [v];
     Lf: free(y);
     y := malloc(); [v] := y)
  + (assume(z = 0); skip)
```

```
client(v)  $\triangleq$ 
  local x in
    x := [v];
    push_back(v);
    Lrx: [x] := 42
```

ISL version

// client, inlining proc call

```
x := [v];
( // push_back
  y := [v];
  free(y);
  y := alloc();
  [v] := y
+
  skip
)
```

[x] := 42

SepSL version

stronger guarantee:
any state in pre can
lead to error

more succinct
post

$\langle v \mapsto a * a \mapsto _ * \text{true} \rangle \text{ rclient } \langle x \not\mapsto _ * \text{true} \rangle$

ISL derivation

$[v \mapsto a * a \mapsto -]$

local y, z in

$z := *; //$ HAVOC

$[ok : z=1 * v \mapsto a * a \mapsto -]$

($\text{assume}(z \neq 0); //$ ASSUME

$[ok : z=1 * z \neq 0 * v \mapsto a * a \mapsto -]$

$L_{rv} : y := [v]; //$ LOAD

$[ok : z=1 * y=a * v \mapsto a * a \mapsto -]$

$L_f : \text{free}(y); //$ FREE

$[ok : z=1 * y=a * v \mapsto a * a \not\mapsto]$

$y := \text{malloc}(); //$ ALLOC1, CHOICE

$[ok : z=1 * v \mapsto a * a \not\mapsto * y \mapsto -]$

$[v] := y; //$ STORE

$[ok : z=1 * v \mapsto y * a \not\mapsto * y \mapsto -]$

) + (...) $//$ CHOICE

$[ok : z=1 * v \mapsto y * a \not\mapsto * y \mapsto -]$

$//$ LOCAL

$[ok : \exists a'. v \mapsto a' * a' \mapsto - * a \not\mapsto]$

$[v \mapsto a * a \mapsto -]$

local x in

$x := [v]; //$ LOAD

$[ok : x=a * v \mapsto a * a \mapsto -]$

$\text{push_back}(v); //$ PB-OK

$[ok : \exists a'. x=a * v \mapsto a' * a' \mapsto - * a \not\mapsto] //$ CONS

$[ok : \exists a'. x=a * v \mapsto a' * a' \mapsto - * x \not\mapsto]$

$L_{rx} : [x] := 88; //$ STOREER

$[er(L_{rx}) : \exists a'. x=a * v \mapsto a' * a' \mapsto - * x \not\mapsto]$

$//$ LOCAL

$[er(L_{rx}) : \exists a'. v \mapsto a' * a' \mapsto - * a \not\mapsto]$

SepSIIL derivation

$\langle v \mapsto a * a \mapsto _ * \text{true} \rangle \Rightarrow \langle v \mapsto a * a \mapsto _ * (a = a \vee a \not\mapsto) * \text{true} \rangle$

$x := [v];$ // Load + Frame

$\langle v \mapsto a * a \mapsto _ * (x = a \vee x \not\mapsto) * \text{true} \rangle \Rightarrow \langle (v \mapsto a * a \mapsto _ * (x = a \vee x \not\mapsto) * \text{true}) \vee (x \not\mapsto * \text{true}) \rangle$

(// push_back: Choice

$\langle v \mapsto a * a \mapsto _ * (x = a \vee x \not\mapsto) * \text{true} \rangle$

$y := [v];$ // Load + Frame

$\langle v \mapsto a * y \mapsto _ * (x = y \vee x \not\mapsto) * \text{true} \rangle \Rightarrow \langle v \mapsto _ * y \mapsto _ * (x = y \vee x \not\mapsto) * \text{true} \rangle$

$\text{free}(y);$ // Free + Frame

$\langle v \mapsto _ * y \not\mapsto * (x = y \vee x \not\mapsto) * \text{true} \rangle \Rightarrow \langle x \not\mapsto * v \mapsto _ * \text{emp} * \text{true} \rangle$

$y := \text{alloc}();$ // Alloc + Frame

$\langle x \not\mapsto * v \mapsto _ * y \mapsto _ * \text{true} \rangle \Rightarrow \langle x \not\mapsto * v \mapsto _ * \text{true} \rangle$

$[v] := y$ // Write + Frame

$\langle x \not\mapsto * v \mapsto y * \text{true} \rangle \Rightarrow \langle x \not\mapsto * \text{true} \rangle$

+

$\langle x \not\mapsto * \text{true} \rangle \text{ skip } \langle x \not\mapsto * \text{true} \rangle$ // Skip + Frame

)

$\langle x \not\mapsto * \text{true} \rangle$

$[x] := 88$

Correctness and completeness

Relational semantics

$$\llbracket \text{skip} \rrbracket \triangleq \{(\sigma, \sigma)\}$$

$$\llbracket b? \rrbracket \triangleq \{(\sigma, \sigma) \mid \sigma = \langle s, h \rangle \wedge s \models b\}$$

$$\llbracket x := a \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto \llbracket a \rrbracket s], h) \}$$

$$\llbracket x := [y] \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto v], h) \mid v = h(s(y)) \in \mathbb{Z}\}$$

$$\llbracket [x] := y \rrbracket \triangleq \{(\langle s, h \rangle, \langle s, h[s(x) \mapsto s(y)] \rangle) \mid h(s(x)) \in \mathbb{Z}\}$$

$$\llbracket x := \text{alloc}() \rrbracket \triangleq \{(\langle s, h \rangle, \langle s[x \mapsto n], h[n \mapsto v] \rangle) \mid v \in \mathbb{Z} \wedge (n \notin \text{dom}(h) \vee h(n) = \perp)\}$$

$$\llbracket \text{free}(x) \rrbracket \triangleq \{(\langle s, h \rangle, \langle s, h[s(x) \mapsto \perp] \rangle) \mid h(s(x)) \in \mathbb{Z}\}$$

Actual rules of SepSIL

$$\frac{}{\langle\langle \mathbf{emp} \rangle\rangle \text{ skip } \langle\langle \mathbf{emp} \rangle\rangle} \langle\langle \text{skip} \rangle\rangle$$

$$\frac{}{\langle\langle q[a/x] \rangle\rangle x := a \langle\langle q \rangle\rangle} \langle\langle \text{assign} \rangle\rangle$$

$$\frac{}{\langle\langle \mathbf{emp} \rangle\rangle x := \text{alloc}() \langle\langle x \mapsto v \rangle\rangle} \langle\langle \text{alloc1} \rangle\rangle$$

$$\frac{}{\langle\langle q \wedge b \rangle\rangle b? \langle\langle q \rangle\rangle} \langle\langle \text{assume} \rangle\rangle$$

$$\frac{}{\langle\langle \beta \not\mapsto \rangle\rangle x := \text{alloc}() \langle\langle x = \beta \wedge x \mapsto v \rangle\rangle} \langle\langle \text{alloc2} \rangle\rangle$$

$$\frac{}{\langle\langle x \mapsto - \rangle\rangle \text{ free}(x) \langle\langle x \not\mapsto \rangle\rangle} \langle\langle \text{free} \rangle\rangle$$

$$\frac{x \notin \text{fv}(a)}{\langle\langle y \mapsto a * q[a/x] \rangle\rangle x := [y] \langle\langle y \mapsto a * q \rangle\rangle} \langle\langle \text{load} \rangle\rangle$$

$$\frac{}{\langle\langle x \mapsto - \rangle\rangle [x] := y \langle\langle x \mapsto y \rangle\rangle} \langle\langle \text{store} \rangle\rangle$$

$$\frac{\langle\langle p \rangle\rangle r \langle\langle q \rangle\rangle \quad \text{fv}(t) \cap \text{mod}(r) = \emptyset}{\langle\langle p * t \rangle\rangle r \langle\langle q * t \rangle\rangle} \langle\langle \text{frame} \rangle\rangle$$

$$\frac{\langle\langle p \rangle\rangle r \langle\langle q \rangle\rangle \quad x \notin \text{fv}(r)}{\langle\langle \exists x.p \rangle\rangle r \langle\langle \exists x.q \rangle\rangle} \langle\langle \text{exists} \rangle\rangle$$

Correctness + (relative) completeness

Th. [*correctness*]

If $\langle P \rangle c \langle Q \rangle$ then $P \subseteq \llbracket c \rrbracket^{\text{op}} Q$

Proof. By induction on the derivation.

Th. [*completeness*]

Any valid triple $\langle P \rangle c \langle Q \rangle$ can be derived

Proof. See OOPSLA2025 paper.

Exercises

Validity

Which SepSIL triples are valid ?

$\langle \text{emp} \rangle \text{ free}(x) \langle x \not\mapsto \rangle$



$\langle x \not\mapsto \rangle \text{ free}(x) \langle x \not\mapsto \rangle$



$\langle \text{false} \rangle \text{ free}(x) \langle \text{emp} \rangle$



$\langle x \mapsto _ \rangle \text{ free}(x) \langle \text{emp} \rangle$



Checking

Validate previous ISL / SepSL derivations

```
[v ↦ a * a ↦ -]
local y, z in
  z := *; // HAVOC
  [ok : z = 1 * v ↦ a * a ↦ -]
  ( assume(z ≠ 0); // ASSUME
    [ok : z = 1 * z ≠ 0 * v ↦ a * a ↦ -]
    Lrv: y := [v]; // LOAD
    [ok : z = 1 * y = a * v ↦ a * a ↦ -]
    Lf: free(y); // FREE
    [ok : z = 1 * y = a * v ↦ a * a ↦ -]
    y := malloc(); // ALLOC1, CHOICE
    [ok : z = 1 * v ↦ a * a ↦ - * y ↦ -]
    [v] := y; // STORE
    [ok : z = 1 * v ↦ y * a ↦ - * y ↦ -]
  ) + (...) // CHOICE
  [ok : z = 1 * v ↦ y * a ↦ - * y ↦ -]
// LOCAL
[ok : ∃a'. v ↦ a' * a' ↦ - * a ↦ - ]

[v ↦ a * a ↦ -]
local x in
  x := [v]; // LOAD
  [ok : x = a * v ↦ a * a ↦ -]
  push_back(v); // PB-OK
  [ok : ∃a'. x = a * v ↦ a' * a' ↦ - * a ↦ -] // CONS
  [ok : ∃a'. x = a * v ↦ a' * a' ↦ - * x ↦ -]
  Lrx: [x] := 88; // STOREER
  [er(Lrx) : ∃a'. x = a * v ↦ a' * a' ↦ - * x ↦ -]
// LOCAL
[er(Lrx) : ∃a'. v ↦ a' * a' ↦ - * a ↦ - ]

⟨v ↦ a * a ↦ _ * true⟩ ⇒ ⟨v ↦ a * a ↦ _ * (a = a ∨ a ↯) * true⟩
x := [v]; // Load + Frame
⟨v ↦ a * a ↦ _ * (x = a ∨ x ↯) * true⟩ ⇒ ⟨(v ↦ a * a ↦ _ * (x = a ∨ x ↯) * true) ∨ (x ↯ * true)⟩
( // push_back: Choice
  ⟨v ↦ a * a ↦ _ * (x = a ∨ x ↯) * true⟩
  y := [v]; // Load + Frame
  ⟨v ↦ a * y ↦ _ * (x = y ∨ x ↯) * true⟩ ⇒ ⟨v ↦ _ * y ↦ _ * (x = y ∨ x ↯) * true⟩
  free(y); // Free + Frame
  ⟨v ↦ _ * y ↯ * (x = y ∨ x ↯) * true⟩ ⇒ ⟨x ↯ * v ↦ _ * emp * true⟩
  y := alloc(); // Alloc + Frame
  ⟨x ↯ * v ↦ _ * y ↦ _ * true⟩ ⇒ ⟨x ↯ * v ↦ _ * true⟩
  [v] := y // Write + Frame
  ⟨x ↯ * v ↦ y * true⟩ ⇒ ⟨x ↯ * true⟩
+
  ⟨x ↯ * true⟩ skip ⟨x ↯ * true⟩ // Skip + Frame
)
⟨x ↯ * true⟩
[x] := 88
```


"Compiling"

Transform the following C-like code in the syntax of SepSIL

$$i := 0 ; q := * p ; \text{while } (q \neq \text{nil}) \text{ do } \{ q := * q ; i := i + 1 \}$$

"Compiling"

Transform the following C-like code in the syntax of SepSL

$i := 0 ; q := * p ; \text{while } (q \neq \text{nil}) \text{ do } \{ q := * q ; i := i + 1 \}$

$i := 0 ;$

$q := [p] ;$

$((q \neq \text{nil}?) ; q := [q] ; i := i + 1)^{\star} ;$

$(q = \text{nil}?)$

Reasoning

Prove the SepSIL triple $\langle p \mapsto \text{nil} * \text{true} \rangle c \langle i = 0 \rangle$ where

$c \triangleq i := 0 ; q := * p ; \text{while } (q \neq \text{nil}) \text{ do } \{ q := * q ; i := i + 1 \}$

$\langle p \mapsto \text{nil} * \text{true} \rangle$

$i := 0 ;$

$q := [p] ;$

$((q \neq \text{nil}?) ; q := [q] ; i := i + 1)^* ;$

$(q = \text{nil}?)$

$\langle i = 0 \rangle$

Reasoning

Prove the SepSIL triple $\langle p \mapsto \text{nil} * \text{true} \rangle c \langle i = 0 \rangle$ where

$c \triangleq i := 0 ; q := * p ; \text{while } (q \neq \text{nil}) \text{ do } \{ q := * q ; i := i + 1 \}$

$\langle p \mapsto \text{nil} * \text{true} \rangle \Rightarrow \langle v = \text{nil} * p \mapsto v \rangle$

$i := 0 ;$

$\langle i = 0 \wedge v = \text{nil} * p \mapsto v \rangle$

$q := [p] ;$

$\langle i = 0 \wedge q = \text{nil} * p \mapsto v \rangle \Rightarrow$

$\langle i = 0 \wedge q = \text{nil} \rangle$

$((q \neq \text{nil}?) ; q := [q] ; i := i + 1)^* ;$

$\langle i = 0 \wedge q = \text{nil} \rangle$

$(q = \text{nil}?)$

$\langle i = 0 \rangle$